

Autenticação JWT em Java com Spring Security + java-jwt (Auth0)

Resumo comentado da configuração para proteger uma API REST Java usando:

- `spring-boot-starter-security`
- `com.auth0:java-jwt:4.4.0`

Fontes consultadas: [Spring Security – Architecture](#), [auth0/java-jwt \(GitHub\)](#), [Baeldung – Managing JWT With Auth0 java-jwt](#), [Auth0 – Java Spring Security Quickstart](#)

1. Visão geral do fluxo

```
Cliente --POST /auth/login (user/senha)--> AuthController
                                         |
                                         AuthenticationManager valida credenciais
                                         |
                                         JwtService gera o token (java-jwt)
                                         v
Cliente <----- { "token": "eyJhbGciOi..." } -----
Cliente --GET /api/recurso Authorization: Bearer <token>--> JwtAuthenticationFilter
                                         |
                                         JwtService valida o token (java-jwt)
                                         |
                                         Preenche o SecurityContextHolder
                                         v
                                         Requisição segue para o Controller
```

Pontos-chave:

- `spring-boot-starter-security` cuida da cadeia de filtros (`SecurityFilterChain`), do `SecurityContextHolder` e da autorização por rota.
 - `java-jwt` cuida apenas de criar e verificar o token em si (assinatura, claims, expiração). Ele não sabe nada sobre Spring — a integração é feita por um filtro customizado.
-

2. `JwtService` — criação e verificação do token (java-jwt)

```
@Component
public class JwtService {

    // NUNCA deixe o segredo hardcoded em produção – use variável de ambiente
    // ou um cofre de segredos (Vault, AWS Secrets Manager, etc.)
    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration-minutes:60}")
    private long expirationMinutes;
```

```

private Algorithm algorithm() {
    // HMAC256 é simétrico: a mesma chave assina e verifica.
    // Se precisar que outro serviço só verifique (sem poder emitir tokens),
    // prefira RSA256 (par de chaves pública/privada) em vez de HMAC.
    return Algorithm.HMAC256(secret);
}

/** Gera o token após autenticação bem-sucedida. */
public String gerarToken(String username, List<String> roles) {
    Instant agora = Instant.now();
    return JWT.create()
        .withSubject(username)           // "sub": identifica o usuário
        .withIssuer("minha-api")         // "iss": quem emitiu o token
        .withClaim("roles", roles)       // claim customizada
        .withIssuedAt(agora)             // "iat"
        .withExpiresAt(agora.plus(expirationMinutes, ChronoUnit.MINUTES)) //
        .sign(algorithm());              // assina e serializa o JWT
}

/**
 * Valida assinatura + claims (issuer, expiração) e retorna o token decodificado.
 * Lança JWTVerificationException se o token for inválido/expirado.
 */
public DecodedJWT validarToken(String token) {
    JWTVerifier verifier = JWT.require(algorithm())
        .withIssuer("minha-api")
        .acceptLeeway(5) // opcional: tolerância (segundos) para relógios
        .build();
    return verifier.verify(token); // lança exceção se inválido
}
}

```

Notas importantes do `java-jwt`:

- `JWT.create()` monta o token com um builder fluente; `.sign(algorithm)` finaliza e assina.
- `JWT.require(algorithm)` cria um `JWTVerifier` reutilizável — construa-o uma vez (ou por requisição) e chame `.verify(token)`.
- Exceções relevantes (todas em `com.auth0.jwt.exceptions`):
 - o `JWTVerificationException` — classe-mãe, cobre qualquer falha de verificação.
 - o `TokenExpiredException` — token expirado (`exp` no passado).
 - o `SignatureVerificationException` — assinatura não confere (chave errada ou token adulterado).
 - o `JWTDecodeException` — token malformatado (nem chegou a decodificar).
- Algoritmos suportados: HMAC (HS256/384/512), RSA (RS256/384/512) e ECDSA (ES256/384/512). Para APIs simples, HS256 com um segredo forte (≥256 bits) é suficiente; use RSA/ECDSA quando outro serviço precisa validar o token sem poder emitir novos.

3. `JwtAuthenticationFilter` — ponte entre java-jwt e Spring Security

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;

    public JwtAuthenticationFilter(JwtService jwtService) {
        this.jwtService = jwtService;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws
ServletException, IOException {

        String header = request.getHeader("Authorization");

        // Sem header "Bearer": deixa passar sem autenticar
        // (a rota pode ser pública, ou vai ser barrada depois pela
authorizeHttpRequests)
        if (header == null || !header.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        String token = header.substring(7); // remove "Bearer "

        try {
            DecodedJWT decoded = jwtService.validarToken(token);

            String username = decoded.getSubject();
            List<String> roles = decoded.getClaim("roles").asList(String.class);

            var authorities = roles.stream()
                .map(SimpleGrantedAuthority::new)
                .toList();

            // Autenticação "pronta" – não precisamos checar senha de novo,
            // o token já prova quem é o usuário.
            var authentication = new UsernamePasswordAuthenticationToken(
                username, null, authorities);

            // Preenche o contexto de segurança para esta requisição/thread
            SecurityContextHolder.getContext().setAuthentication(authentication);

        } catch (JWTVerificationException e) {
            // Token inválido/expirado: NÃO autentica e segue o filtro.
            // A authorizeHttpRequests vai barrar como 401/403 mais adiante.
            SecurityContextHolder.clearContext();
        }
    }
}
```

```
        filterChain.doFilter(request, response);
    }
}
```

Pontos-chave:

- `OncePerRequestFilter` garante que o filtro rode uma única vez por requisição, mesmo em forwards internos.
- O filtro não lança exceção 401 diretamente — ele apenas não autentica; quem decide o que fazer com uma requisição não autenticada é a configuração de autorização (`authorizeHttpRequests`) e o `AuthenticationEntryPoint`.

4. `SecurityConfig` — registrando tudo na `SecurityFilterChain`

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    public SecurityConfig(JwtAuthenticationFilter jwtAuthenticationFilter) {
        this.jwtAuthenticationFilter = jwtAuthenticationFilter;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            // API stateless: sem cookies de sessão, então CSRF não se aplica
            .csrf(csrf -> csrf.disable())

            // Nunca criar/usar HttpSession – cada requisição se autentica via token
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))

            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/login", "/auth/register").permitAll()
                .requestMatchers("/actuator/health").permitAll()
                .anyRequest().authenticated()
            )

            // Insere nosso filtro ANTES do filtro padrão de login por
            // formulário/usuário-senha,
            // já que quem autentica aqui é o token, não um POST de formulário.
            .addFilterBefore(jwtAuthenticationFilter,
                UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        // BCrypt: usado só no login (comparar senha enviada com o hash salvo)
    }
}
```

```

        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration
config) throws Exception {
        // Necessário para o AuthController conseguir validar usuário/senha no login
        return config.getAuthenticationManager();
    }
}

```

Pontos-chave:

- `SessionCreationPolicy.STATELESS` é essencial em APIs JWT: sem isso o Spring ainda tentaria manter `HttpSession`, o que é redundante e quebra a escalabilidade horizontal.
- `csrf().disable()` só é seguro porque a API é stateless e sem cookies — CSRF explora sessões autenticadas por cookie, que não existem aqui.
- A ordem do `addFilterBefore` importa: o filtro JWT precisa rodar antes do ponto em que o Spring decide se a requisição está autenticada.

5. `AuthController` — emitindo o token no login

```

@RestController
@RequestMapping("/auth")
public class AuthController {

    private final AuthenticationManager authenticationManager;
    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    public AuthController(AuthenticationManager authenticationManager,
                          JwtService jwtService,
                          UserDetailsService userDetailsService) {
        this.authenticationManager = authenticationManager;
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest req) {
        try {
            // Aqui o Spring Security valida usuário/senha (usando o PasswordEncoder)
            authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(req.username(),
req.senha()));
        } catch (BadCredentialsException e) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Credenciais
inválidas");
        }

        UserDetails user = userDetailsService.loadUserByUsername(req.username());
        List<String> roles = user.getAuthorities().stream()

```

```

        .map(GrantedAuthority::getAuthority)
        .toList();

        String token = jwtService.gerarToken(user.getUsername(), roles);
        return ResponseEntity.ok(new TokenResponse(token));
    }

    record LoginRequest(String username, String senha) {}
    record TokenResponse(String token) {}
}

```

6. `application.yml` — configuração do segredo

```

jwt:
  secret: ${JWT_SECRET}           # defina via variável de ambiente, nunca no código-
  fonte
  expiration-minutes: 60

```

7. Boas práticas e armadilhas comuns

Tema	Recomendação
Segredo (HMAC)	Gerar aleatoriamente, ≥ 256 bits, fora do código-fonte (env var / secrets manager)
Expiração	Tokens de acesso curtos (15–60 min); usar refresh token separado para renovar sem exigir login de novo
Claims sensíveis	Nunca colocar senha ou dados sigilosos no payload — o JWT é apenas assinado, não criptografado; qualquer um pode decodificar o conteúdo (é só Base64)
Revogação	JWT é stateless por natureza — para permitir logout/revogação imediata, mantenha uma blacklist (Redis, por exemplo) dos tokens invalidados até a expiração
Algoritmo	Fixe o algoritmo esperado no verificador (<code>JWT.require(algorithm)</code>); não confie em um campo <code>alg</code> vindo do próprio token (evita ataques de "algorithm confusion")
HTTPS	Sempre transmitir o token por HTTPS — em texto puro ele equivale a uma senha
Erros de autenticação	Trate <code>JWTVerificationException</code> e credenciais inválidas retornando 401, nunca vazando detalhes internos (stack trace, etc.)

8. Dependências usadas (referência)

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>com.auth0</groupId>

```

```
<artifactId>java-jwt</artifactId>  
<version>4.4.0</version>  
</dependency>
```

Sources

- [Spring Security – Architecture \(docs.spring.io\)](https://docs.spring.io/spring-security/reference/)
- [auth0/java-jwt — GitHub](https://github.com/auth0/java-jwt)
- [Baeldung – Managing JWT With Auth0 java-jwt](#)
- [Auth0 – Java Spring Security Quickstart](#)